Django Views, Templates, and Forms

16. What is the difference between function-based views (FBVs) and class-based views (CBVs)?

    FBV (Function Based Views) are simple Python functions that take a request and return a response.CBV (Class Based Views) are written as classes and provide more structure and reusability.FBVs are easier to write for small views.CBVs allow using built-in generic views like ListView, DetailView.CBVs support inheritance and make code cleaner for complex apps.

17. Write a simple function-based view that returns "Hello, Django!" as an HTTP response.

```
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello, Django!")
```

18. How do you render a template in Django using the render() function?

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

    First, we import render from django.shortcuts. Then, we create a view function (like home). Inside it, we call render(request, 'home.html'). request is the incoming HTTP request. 'home.html' is the template we want to show.

19. Explain how context data is passed from a view to a template.

```
from django.shortcuts import render

def home(request):
    context = {'name': 'Raksha', 'age': 21}
    return render(request, 'home.html', context)
```

    In the view, we create a dictionary with data like {'name': 'Raksha', `age`:21}. This dictionary is passed to render(). Inside the template, we can access it using {{ name }}. This allows dynamic content display.

20. What are template tags and filters? Give two examples.

    Template tags are special keywords used for logic in templates (like loops, conditions).

    Example: {% for book in books %} {{ book }} {% endfor %}.

    Filters are used to modify variables.

    Example: {{ name|upper }} → converts name to uppercase.

Creating and Handling Views for User Interactions

21. How do you handle GET vs POST requests inside a Django view?

In Django views, we check if request.method == 'GET' for GET requests. For POST, we check if request.method == 'POST'. GET is usually used to fetch data, POST is used to submit data like forms.

22. Write a Django view that handles a contact form submission and returns a success message.

```
from django.http import HttpResponse

def contact(request):
    if request.method == "POST":
        name = request.POST.get('name')
        email = request.POST.get('email')
        message = request.POST.get('message')
        return HttpResponse("Thank you, your message has been sent successfully!")
```

23. How do you implement redirecting users to another page in Django?

We use redirect('url_name') function in views. Example: return redirect('home'). It sends the user to another page.

```
from django.shortcuts import redirect

def my_view(request):
    return redirect('home')
```

24. What is the difference between HttpResponse and JsonResponse in Django?

HttpResponse is used to send normal HTML/text responses. JsonResponse is used when we want to return data in JSON format. JsonResponse is mostly used in APIs and AJAX calls.

25. What is the use of request.POST and request.GET dictionaries in views?

- request.GET is a dictionary containing data from URL query parameters.
- request.POST is a dictionary containing data submitted by forms.
- Example: request.GET['id'], request.POST['username'].

Working with Django Templates for Dynamic Content

26. How do you include one template inside another in Django (e.g., base.html with {% block %})?

Django allows us to make a base template (like base.html) which has the common design (header, footer, CSS).Inside the base template, we define {% block %} tags where child templates can insert their content.Then, in another template, we use {% extends 'base.html' %} to use the base file.In that child template, we write {% block content %} ... {% endblock %} to put custom content.

base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My Site</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    {% block content %}{% endblock %}
    <footer>© 2025 My Site</footer>
</body>
</html>
```

home.html

```
{% extends 'base.html' %}

{% block content %}
    <p>This is the home page content.</p>
{% endblock %}
```

27. Explain how the {% for %} and {% if %} tags work in templates. Give an example.

- {% for %} is used to loop over items in a list.

Example:

```
{% for book in books %}
  <p>{{ book }}</p>
{% endfor %}
```

- {% if %} is used for conditions.

Example:

```
{% if user.is_authenticated %}
  <p>Welcome {{ user.username }}</p>
{% endif %}
```

28. What is the purpose of {% static %} in Django templates?

{% static %} is used to load static files like CSS, JS, and images in templates.

Example: <link rel="stylesheet" href="{% static 'css/style.css' %}">. It tells Django to fetch files from the static folder.

29. How do you pass query results from models to templates for rendering dynamic content?

    In views, we get data from the model like books = Book.objects.all(). Pass it in context dictionary: {'books': books}. In template, use a loop {% for book in books %} {{ book.title }} {% endfor %}.

30. Write a template snippet that displays a list of book titles dynamically from a context variable books.

```
<ul>
  {% for book in books %}
    <li>{{ book.title }}</li>
  {% endfor %}
</ul>
```